Minimized Interference in Multi-Task Networks via Structured Routing and Static Feature Extractors

Rajiv Prakash[1]*, Julia Meyer[2]†, Yusuke Nakamoto[2], Marta Dos Santos[2], and Fatemeh Sharifzadeh[2]

[1] Indian Institute of Technology, Delhi
[2] Max Planck Institute for Intelligent Systems, Tübingen

*rajiv.prakash@iitd.ac.in; †julia.meyer@tuebingen.mpg.de

Abstract

Multi-task learning (MTL) aims to consolidate multiple tasks within a single model by capitalizing on beneficial information sharing. Nevertheless, significant challenges arise from negative interference between tasks, often reducing overall model performance. Prior efforts to address these challenges have typically leveraged dynamic loss balancing, regularized parameter sharing, or implicit, partially-overlapping architecture partitions. This work introduces Partitioned Pathways and Non-Adaptive Primitives (PP-NAP), a framework that employs explicit feature routing in tandem with fixed, non-adaptive primitive units to minimize crosstalk between tasks. Rather than relying solely on optimization, the model statically defines task-agnostic, non-learnable primitive operators for basic feature extraction, following these with explicit routing layers that separate shared and task-specific parameter groups. Rigorous evaluations across multiple visual domains, including semantic segmentation and multi-label classification, demonstrate that PP-NAP networks provide substantial improvements over established baselines, both in terms of accuracy and efficiency. The fully explicit parameter allocation allows the model to leverage commonalities while safeguarding against harmful interference, and achieves parameter savings with negligible loss in expressive capability. Empirical comparisons across diverse multi-task benchmarks reveal that the proposed strategy yields robust, scalable learning with reduced computational demand, contributing a systematic, easily-deployable solution to the persistent problem of interference in multi-task neural models.

1. Introduction

Multi-task learning (MTL) is commonly employed to improve learning efficiency and performance of multiple tasks by using supervised signals from other related tasks [6, 33, 49]. These models have led to impressive results across numerous tasks. However, there is well-documented evidence [18,28,41,53] that these models are suffering from *task interference* [53], thereby limiting multi-task networks (MTNs) from realizing their full potential.

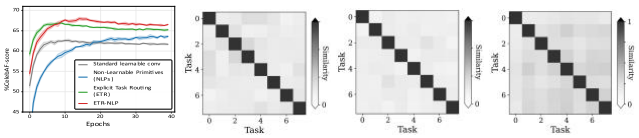


Figure 1. (a) Learning progression of multi-task networks (MTNs) on CelebA for eight tasks. Hard-sharing models with fully learnable parameters (gray) learn rapidly and then suffer from performance degradation due to conflicting gradients from task interference. Networks with non-learnable primitives (NLPs; blue) do not suffer from task interference by design, while explicit task routing (ETR; green), and ETR with NLPs (red) do not eliminate but suffer less from task interference. (b) Gradient correlations measured via CKA [19] across all pairs of tasks for different layers of a standard MTN at the end of training. Observe the acute lack of correlation between tasks (low off-diagonal magnitude).

For instance, consider the learning progression of an MTN with a standard learnable convolutional layer in Figure 1a (blue curve). Observe that the model learns rapidly, we posit, by exploiting all the shared information between the tasks, i.e., gradients pointing in similar directions. However, the performance starts degrading on further training since the model needs to exploit dissimilar information between the tasks for further improvement, i.e., gradients point in different directions. The latter can be verified by observing the similarity (centered kernel alignment [19]), or the lack thereof, between the gradients for each pair of tasks in Figure 1b.

Several approaches were proposed for mitigating task interference in MTNs, including loss/gradient balancing [17, 21,22,34,52], parameter partitioning [2,28,30,37] and architectural design [8,18,29]. Despite the diversity of these approaches, they share two common characteristics, (i) all parameters are learned, either for a pre-trained task or for the multiple tasks at hand, (ii) the learned parameters are either fully shared across all tasks or are shared across a partial set of tasks through implicit partitioning, i.e., with no di-
rect control over which parameters are shared across which tasks. Both of these features limit the flexibility of

existing multi-task network designs from mitigating the deleterious effects of task interference on their predictive performance.

Relaxing the above design choices is the primary goal of this paper. We propose two complementary design principles, namely *explicit task routing (ETR)*, and *non-learnable primitives (NLPs)*, that explicitly seek to mitigate task interference. Through extensive empirical evaluation, we demonstrate that these two complementary ideas, individually and jointly, help mitigate task interference and consistently improve the performance of MTNs. As can be observed in Figure 1a, compared to a hard-sharing MTN with a standard learnable convolutional layer (gray curve), an MTN with NLP (blue curve) has better learning characteristics, i.e., learn more steadily and not suffer from performance degradation. Similarly, MTN with ETR (green curve) and MTN with ETR-NLP (red curve) does not eliminate task interference but reduce it to an extent. Figure 2 shows an overview of the proposed ETR-NLP networks.

From a network topological perspective, we propose explicit task-routing (ETR), a parameter allocation strategy that partitions the parameters into *shared* and *task-specific* branches. More explicitly, it comprises one branch shared across all tasks and task-specific branches, one for each task. Unlike existing parameter partitioning methods, ETR is designed to offer precise and fine-grained control over *which* and *how many* parameters are *shared* or *not shared* across the tasks. Additionally, ETR is flexible enough to allow existing implicit parameter partitioning methods [30,37] to be incorporated into its shared branch.

From a layer design perspective, we propose using nonlearnable primitives (NLPs) to extract task-agnostic features and allow each task to choose optimal combinations of these features adaptively. There is growing evidence that features extracted from NLPs can be very effective for single-task settings, including for image classification [15,31,44,47,48], reinforcement learning [9] and modeling dynamical systems [27]. NLPs are attractive for mitigating task interference in MTL. Since they do not contain learnable parameters, the task-agnostic features extracted from such layers alleviate the impact of conflicting gradients, thus implicitly addressing task interference. However, the utility and design of NLPs for multi-task networks have not been explored. We summarize our key contributions below:

– We introduce the concept of *non-learnable primitives (NLPs)* and *explicit task routing (ETR)* to mitigate task interference in multi-task learning. We systematically study the effect of different design choices to determine the optimal design of ETR and NLP.

– We demonstrate the effectiveness of ETR and NLP through MTNs constructed with only NLPs (MTN-NLPs) and only ETR (MTN-ETR) for both image-level classification and pixel-level dense prediction tasks.

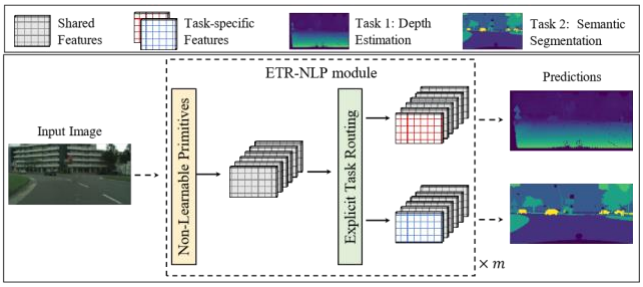


Figure 2. ETR-NLP Networks comprise non-learnable primitives to extract diverse task-agnostic features, followed by explicit task routing to control the parameters/features that are shared across all tasks and those that are exclusive to every single task.

– We evaluate the effectiveness of ETR-NLP networks across three different datasets and compare them against a wide range of baselines for both image-level classification and pixel-level dense prediction tasks. Results indicate that ETR-NLP networks consistently improve performance by a

significant amount.

2. Related Work

We briefly review prior work on non-learnable primitives and mitigating task interference for multi-task learning (MTL), from which our work draws inspiration. Due to space constraints, we refer the readers to the supplementary material for more discussion of related work. We also encourage readers to refer to multiple excellent reviews [3,41,49] for a comprehensive overview of MTL.

Non-Learnable Primitives (NLPs): The notion of NLP for feature extraction was explored for single-task learning motivated either by scientific curiosity or in the quest for computational efficiency [4, 9, 15, 44, 47]. Xu et al. used nonlearnable sparse binary convolutional filters referred to as *LBConv* [15]. Wu et al. proposed randomly adjusting the spatial alignments of data, referred to as *shift* [44]. Xu et al. applied non-learnable additive noises sampled from a uniform distribution to data, referred to as *perturbation* [47]. Yu et al. replaced the attention-based token mixer with a non-parametric pooling primitive in vision transformers [48]. As a common practice, a follow-up 1×1 convolution was used to learn a weighted linear combination of features extracted by non-learnable primitives. These methods generally perform as well as those with standard learnable layers but with much fewer

parameters required to optimize. However, the utility of NLPs for MTL is yet to be explored.

The NLP-based feature extraction proposed in this work is notably different in three key respects: (i) We expand the scope of NLP from single-task image classification to MTL, including image-level and pixel-level prediction tasks. (ii) We consider multiple types of NLPs (i.e., pooling, shift, perturbation, convolution without learnable weights) and demonstrate that a single type of NLP does not benefit MTL. (iii) We design an MTL-specific NLP by exploring various combinations of NLPs under a diverse set of hyperparameters (e.g., pooling/kernel size, sparsity, perturbation strength, real, binary, depth-wise separable weights, etc.).

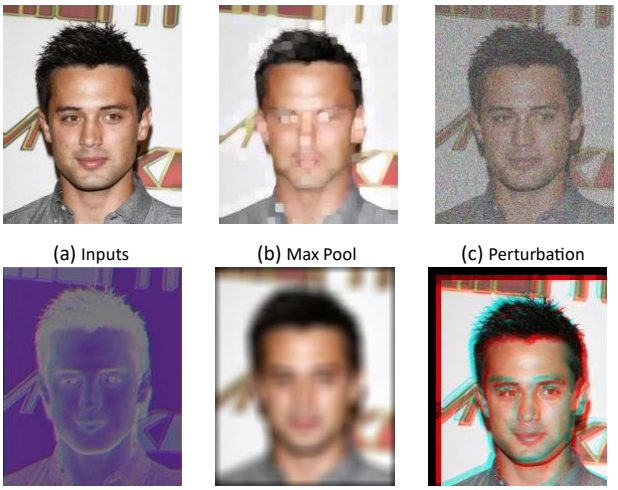
Task Interference in MTL: The success of MTL in computer vision has led to many solutions for mitigating task interference in MTL. The approaches fall into three main categories, (i) loss/gradient balancing [17, 21, 22, 34, 52], (ii) parameter partitioning [2,28,30,37], and (iii) architectural design [8, 18, 29] (supplementary materials). A brief overview is provided below.

–*Loss/Gradient balancing:* Kendall *et al.* [17] utilized homoscedastic uncertainty as task-dependent weights to balance the losses of various tasks. Chen *et al.* [52] adaptively balanced the training of deep MTL models by dynamically adjusting the magnitudes of gradients computed w.r.t different tasks. Liu *et al.* [22] proposed task-specific loss functions to maintain balance among tasks. Finally, Sener and Koltun [34] applied multi-objective optimization to find Pareto-optimal gradients for multiple tasks. The primary goal of this class of methods is to control the contribution of the loss/gradient of each task to the overall loss/gradient, which in turn implicitly helps mitigate task interference.

–*Parameter partitioning:* Attention mechanisms have been widely used to allow networks to focus on different regions of the feature maps adaptively [43]. Attention mechanisms have been employed [22] for MTL at the filter level, allowing each task to select a subset of parameters (i.e., partition) at each layer. Maninis *et al.* [28] used a task-specific squeeze-and-excitation module (i.e., channel attention) for soft parameter partitioning. Strezoski *et al.* [37] introduced a task routing module as a hard parameter partitioning strategy to alleviate interference among tasks by randomly assigning a sub-network to each task. Once assigned, the parameter partitioning remained unchanged. Maximum roaming improved task routing by adaptively updating the parameter partition assignments during training [30]. In contrast to the overlapped parameter partitioning in the aforementioned work, our task routing strategy explicitly reserves a task-specific branch of parameters exclusive for each task, leading to more precise control over the partitioning of parameters among the tasks.

3. ETR-NLP Network Design

We first introduce non-learnable primitives (NLPs) based feature extraction and explicit task routing (ETR). Then, we integrate both into a single module, dubbed ETRNLP, that can be incorporated into modern MTL architectures (e.g., ResNets [13], VGGs [36], SegNet [1], etc.) in a straightforward manner. Lastly, we describe the network's training and inference process with ETR-NLP modules.



(a) Inputs (b) Max Pool (c) Perturbation

(d) Conv (binary) (e) Avg. Pool (f) Shift

Figure 3. Visualization of features extracted by various NLPs: Given an input image (a), different types of NLP extract diversely different features (b) - (f).

3.1. NLP based Feature Extraction

NLPs afford several attractive properties that render them well-suited for MTL. *Primarily*, NLPs do not have any learnable parameters. Hence, the extracted features are agnostic to any particular task, alleviating the impact of disparate gradients. As such, NLPs implicitly mitigate task interference and lead to better predictive performance. A *secondary benefit* is from a computational perspective. Our proposed NLP design offers computation benefits in terms of fewer learned parameters or lower FLOPs. However, as we demonstrate in §4.2.1, obtaining parameter efficiency in MTL is challenging since directly employing existing efficiency-oriented convolutional layer designs (which work very well on standard problems) leads to performance loss on MTL.

We hypothesize that extracting features from nonlearnable primitives (NLPs) that are neither biased nor adaptable to the tasks at hand can mitigate task interference in MTL. A plethora of NLPs are available,

including both non-parametric (e.g., average/max pooling, identity mapping, etc.) and weight-agnostic ones (e.g., LBConv [15], perturbation [47], shift [4,44], etc.). Furthermore, one can tune each type of NLP by adjusting its hyperparameters, such as pooling size, perturbation strength, the sparsity of the non-learnable weights, etc.

However, we demonstrate in Table 1 directly employing a single type of non-learnable primitive degrades the performance of the corresponding MTL model. Since different tasks benefit from different kinds of features, a single NLP is sub-optimal for MTL. And, as we observe in Figure 3, different NLPs extract different features. Therefore, to facilitate extracting a dictionary of diverse features, we place various types of NLPs across different hyperparameter combinations in parallel, similar to an Inception structure [38].

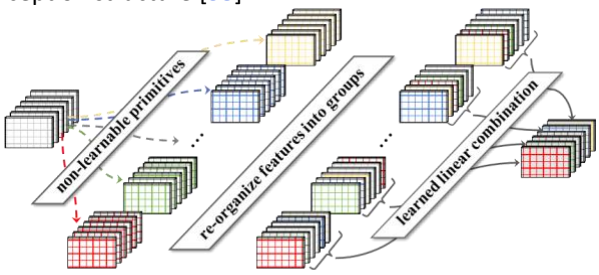


Figure 4. **Design of our NLP-based feature extraction:** It first uses various types of non-learnable primitives (e.g., pooling, shift [4, 44], perturbation [47], etc.) with different hyperparameters (e.g., kernel size, sparsity, perturbation strength, etc.) to extract a dictionary of diverse features. Then, these features are re-arranged into groups by taking one output feature from each primitive. Subsequently, a linear combination is learned to compress features in a group into a single output feature. Finally, these compressed features from different groups are concatenated together.

Next, we re-arrange the extracted features into groups to enhance diversity by taking one feature map from each primitive. Then, a linear combination among features within a group is learned via group-wise 1×1 convolutions. A pictorial illustration of this process is provided in Figure 4.

Based on the schema described above, we first determine the optimal combination of NLPs for MTL. Specifically, we consider five types of NLPs, i.e., average pooling, max pooling, convolution with fixed weights [15], shift [44], and perturbation [47], for a total of $\sum_{i=1}^{5} \binom{5}{i} = 31$ possible variations. Then, we evaluate each variation on both CelebA multi-attribute classification [23] and Cityscapes dense prediction (semantic segmentation and depth estimation) [5] MTL problems, and perform five repetitions to account for performance fluctuations. A representative subset of results is presented in Table 1 (see supplementary material for full results). We observed that using more NLPs for extracting features generally leads to better MTL performance. In particular, the combination of averaging pool, convolution with non-learnable weights, and perturbation in parallel emerges as the top choice, i.e., our final configuration of NLPs. The effects of the hyperparameters of NLPs are presented in §5.

3.2. Explicit Task Routing (ETR)

Despite the extracted features being agnostic to any particular task, a standalone application of NLPs does not proactively address task interference. Therefore, to complement NLP-based feature extraction, we present a novel parameter partitioning method, dubbed *explicit task routing* (ETR), to provide precise and fine-grained control over the sharing of parameters among tasks.

Figure 5 provides a pictorial illustration of ETR (along with the de-facto hard parameter sharing and existing parameter partitioning methods [30,37]) for a three-task sce-

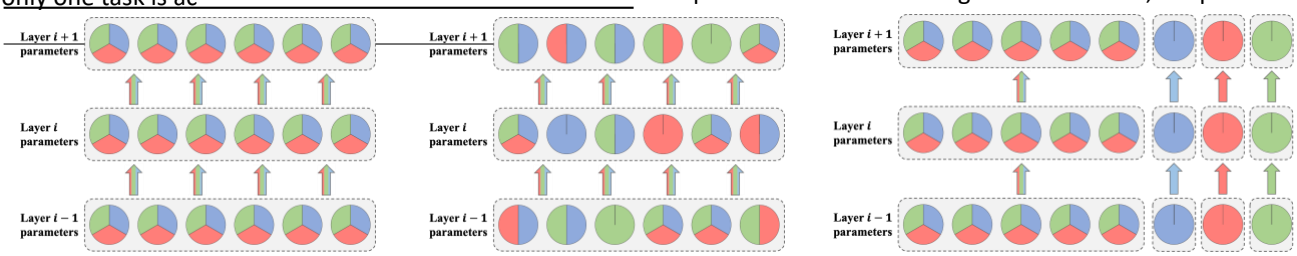
Table 1. Effect of different configurations of NLPs: Relative improvements/degradation over standard learnable convolution on CelebA multi-attribute classification are highlighted in color.

| #Types | Non-Learnable Primitives | | |

standard hard parameter sharing MTL. The effect of γ is studied in §4.2.2.

3.3. ETR-NLP Module

We incorporate the proposed NLPs-based feature extraction and ETR parameter partitioning into a single module, dubbed ETR-NLP, and form MTL networks by replacing the standard convolutional layers in modern MTL architectures with ETR-NLP modules. In each ETR-NLP module, since interactions between the tasks happen through the shared branch, the shared features are obtained by recombining task agnostic features through learned via groupwise 1×1 convolutions. Furthermore, since the taskspecific weights are exclusive to each task and do not interfere with other tasks, the task-specific features are obtained by directly applying standard 3×3 convolutions to features from the previous layer, i.e., task-specific features are not extracted for the task-specific branch. A pictorial illustration of ETR-NLP and its corresponding pseudocode is shown in Figure 1 and Algorithm 1, respectively. Training and inference: Similar to prior parameter partitioning-based methods [28,30,37], only one task is activated at a time during a forward pass of our ETR-NLPbased MTL networks. The training process for ETR-NLPbased networks proceeds as follows. The shared branch and one task-specific branch are activated during a forward pass. As shown in Figure 5c, when task i ($i \in [1,T]$) is active, features for the i-th task will be extracted through the shared and the active i-th task-specific parameters. After training the current task, the parameters of the shared branch are updated immediately for image-level classification MTL problems (e.g., CelebA). While for dense prediction MTL problems (e.g., Cityscapes, NYU-v2), we wait until all tasks are forwarded before updating the parameters of the shared branch. These decisions are driven by an ablative analysis shown in §5. During inference, a separate per-task evaluation is required as the input propagates through the shared and task-specific branches.



(a) Hard parameter sharing (b) Existing parameter partitioning methods (c) Parameter partitioning through explicit task routing (ours)

Figure 5. Illustration of Explicit Task Routing (ETR): (a) The de-facto hard parameter sharing method assigns all parameters to all tasks. (b) Existing parameter partitioning methods assign a partial and overlapped set of parameters for each task, and the assignment is either kept fixed [37] or updated/learned iteratively [28,30]. (c) Our task explicit task routing separates parameters into a common branch shared by all tasks and a task-specific branch reserved exclusively for each task, providing more precise control over parameter partitioning among tasks. Note that only one task is activated during a forward pass for both the existing and our proposed parameter partitioning methods.

Algorithm 1 ETR-NLP: PyTorch-like Pseudocode

```
# C_in, C_out: number of input/output channels
# gamma: sharing ratio of explicit task routing
# prims: settings of non-learnable primitives
# T: No. of tasks
class NLP(nn.Module):
    def __init__(self, C_in, C_out, prims, **kwargs): # define non-learnable
        primitives for i, op in enumerate(prims):
                                self.add_module(str(i), op(C_in, **kwargs))
        k = len(prims) # No. of NLPs # group-wise linear combination
        conv1x1 = nn.Conv2d(C_in * k, C_out, ks=1, groups=C_in)

    def forward(self, x): # extract features by NLPs y = torch.cat([op(x) for
        i, op in enumerate( self.values())], dim=1) # re-arrange features
        via channel shuffling y = torch.channel_shuffle(y, groups=k)
        return conv1x1(y)

class ETR-NLP(nn.Module): def __init__(self, C_in, C_out, gamma, prims, T,
        **kwargs):
        C_shared = int(gamma * C_out)
        C_specif = int(C_out - self.C_shared) # define a shared branch
        shared_branch = NLP(C_in, C_shared, prims, **
            kwargs) # define task-specific branches
        for i in range(T):
            specif_branch = nn.Conv2d(C_in, C_specif, ks=3, s=1, p=1) #
            standard Conv self.add_module("task_{}".format(i),
                specif_branch)
        self.task = 0 # set an active task

    def get_layer(self, name): return getattr(self,
        name)

    def forward(self, x): shared = shared_branch(x) specif =
        self.get_layer("task_{}".format( self.task))(x)
        return torch.cat([shared, specif], dim=1)
```

4. Experimental Evaluation

In this section, we first describe our experimental setup. Then, we independently demonstrate the effectiveness of NLPs and ETR for MTL. Finally, we compare our ETRNLP to

a wide range of MTL baselines for both image-level classification and pixel-level prediction problems.

4.1. Experimental Setup

Datasets. We conduct experiments on three widely used MTL benchmarks: *CelebA* [23] is a large-scale face attributes dataset containing more than 200K celebrity images, each with 40 binary attribute annotations that can be grouped into eight categories. Accordingly, we can define an eight or 40-task MTL problem by considering each group or attribute as an individual binary classification task. *Cityscapes* [5] is a large-scale dataset for the semantic understanding of urban street scenes. It is split into training, validation, and test sets, with 2975, 500, and 1525 images. Following [22,30], we resize all images to 128 by 256 and use the median level segmentation comprising seven semantic categories. Together with depth estimation, we define an eight-task MTL problem by treating the segmentation of each semantic category separately. *NYU-v2* [35] is a video sequence dataset composed of 1449 indoor images recorded over 464 scenes from a Microsoft Kinect camera. Following [22], we resize all images to 288 by 384 resolution. It supports the segmentation of 13 semantic categories, depth estimation, and surface normal estimation for 15 tasks. More details are available in the supplementary.

Implementation Details. We implement ETR-NLP in ResNet18 [13] and VGG16 [36] architectures for imagelevel classification problems (e.g., CelebA), and in SegNet [1] architecture for pixel-level dense prediction problems (e.g., Cityscapes and NYU-v2). For training on CelebA, we use Adam optimizer with a learning rate of 10^{-4} and a batch size of 256 images for 40 epochs. For training on Cityscapes and NYU-v2, we also use Adam optimizer with a learning rate of 10^{-4}, but with a batch size of 8 images for 500 epochs. We repeat each experiment five times.

Evaluation Metrics. To evaluate the performance on image-level classification MTL problems, we consider *precision*, *recall*, and *F-Score*. *Precision* measures how precise a method is regarding how many predicted true instances are true positives. *Recall* estimates how well a method has adapted to each task by measuring how much of the actual positive instances are recognized. *F-Score* provides a composite measurement derived from precision and recall. This work reports mean precision, recall, and F-Score averaged over all tasks. For evaluating the performance on pixel-level dense prediction MTL problems, we track the *mean Intersection over Union* (mIoU), and *pixel accuracy* (Pix. Acc.) averaged over all segmentation tasks, and the *mean absolute* (Abs. Err.) and *relative error* (Rel. Err.) for the depth estimation task. Lastly, following prior work [3,20,41,49], we also report the average relative improvement Δ_p (defined below) w.r.t. a chosen baseline.

$$\Delta_p = 100\% \times \frac{1}{T}\sum_{t=1}^{T}\frac{1}{N_t}\sum_{n=1}^{N_t}(-1)^{p_{t,n}}\frac{(M_{t,n}-M_{t,n}^{\text{baseline}})}{M_{t,n}^{\text{baseline}}}$$

where N_t is the number of metrics in task t, $M_{t,n}$ is the performance of a task balancing method for the n-th metric in task t, $M_{t,n}^{\text{baseline}}$ is defined similarly for the baseline method, and $p_{t,n}$ is one if a higher value indicates better performance for the n-th metric in task t and zero otherwise.

4.2. Experimental Results

4.2.1 Effectiveness of Non-Learnable Primitives

Table 2 compares our proposed NLPs with other alternative operations. We make the following observations, (i) All standalone instantiations of NLPs lead to performance degradation. The lack of diversity in the features is detrimental to predictive performance. (ii) Unlike standalone NLPs, our proposed NLP-based networks achieve higher precision and recall while requiring fewer learnable parameters and FLOPs. Since NLPs extract task-agnostic and diverse features, they can prevent the network parameters from being dominated by one or more tasks and mitigate mutual interference between tasks. Compared to the base-

Table 2. Comparison of NLPs with alternative designs on CelebA image-level classification problems. 2× means a width multiplier of 2. Our results are highlighted with shading.

	Method	#P	#F	Prec. (↑)	Recall (↑)	Δ_p (↑)
ResNet18	Conv	11.2M	148M	$67.7_{\pm0.8}$	$59.8_{\pm0.3}$	0.0%
	LBConv [15]	1.61M	165M	$65.1_{\pm0.8}$	$53.2_{\pm0.4}$	-7.4%
	Shift [4]	2.81M	42M	$67.5_{\pm1.1}$	$58.4_{\pm0.6}$	-1.3%
	Depth-wise Conv [14]	2.91M	45M	$65.7_{\pm0.4}$	$51.5_{\pm0.4}$	-8.4%
	Ghost module [12]	5.77M	46M	$67.6_{\pm1.5}$	$57.9_{\pm0.6}$	-1.7%
	NLPs	2.05M	43M	**$72.8_{\pm0.3}$**	$59.2_{\pm0.4}$	+3.3%
	NLPs (2×)	8.11M	148M	$71.3_{\pm0.6}$	**$62.2_{\pm0.1}$**	+4.7%
VGG16	Conv	14.7M	1.25G	$71.1_{\pm0.8}$	$63.8_{\pm0.6}$	0.0%
	LBConv [15]	1.84M	1.43G	$67.1_{\pm0.6}$	$58.0_{\pm0.4}$	-7.4%
	Shift [4]	1.67M	0.14G	$68.9_{\pm0.2}$	$59.5_{\pm0.2}$	-4.9%
	Depth-wise Conv [14]	3.57M	0.34G	$65.8_{\pm0.7}$	$51.6_{\pm0.8}$	-13.3%
	Ghost module [12]	7.45M	0.32G	$68.8_{\pm0.8}$	$61.6_{\pm0.4}$	-3.3%
	NLPs	2.48M	0.22G	**$74.2_{\pm0.8}$**	$64.5_{\pm1.1}$	+2.7%
	NLPs (2×)	14.3M	1.23G	$72.5_{\pm0.7}$	**$68.7_{\pm0.9}$**	+4.8%

line architecture with standard learned convolution, NLPsbased networks significantly improve performance.

4.2.2 Effectiveness of Explicit Task Routing

Figure 6 shows the effect of sharing ratio γ on explicit task routing (ETR) performance over the CelebA and Cityscapes datasets. The results show that γ = 0.9, i.e., 90% of the features are shared among tasks, leads to the best performance across both image-level and pixel-level tasks. These results suggest that both cases benefit by sharing a significant number of parameters while still needing a small fraction of task-specific parameters. It is worth noting that when γ = 1, i.e., all tasks share all the parameters of the multitask network, the performance is impaired due to mutual interference between tasks. Accordingly, we set γ = 0.9
for all experiments shown in the main results section.

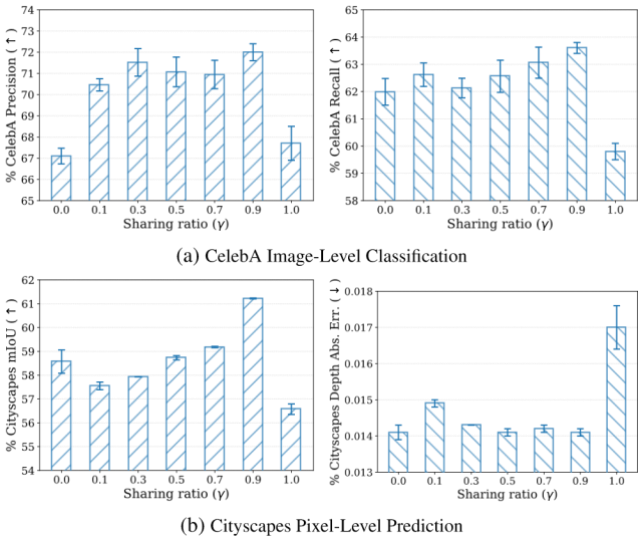


Figure 6. Effects of sharing ratio γ on image-level classification and pixel-level prediction MTL problems.

Table 3. Comparison of ETR with baselines on image-level classification and pixel-level prediction MTL problems. Our results are highlighted with shading.

(a) Image-Level Classification

	Method (ResNet18)	#P (M)	Prec. (↑)	Recall (↑)	Δ_p (↑)
CelebA	Hard sharing	11.2	67.7±0.8	59.8±0.3	0.0%
	Attentive hard sharing [28]	12.9	71.1±0.3	62.6±0.5	+4.9%
	Task routing (γ =0.9) [37]	11.2	71.7±0.1	61.7±0.5	+4.5%
	Max roaming (γ =0.9) [30]	11.2	71.2±0.4	63.0±0.6	+5.3%
	ETR (γ =0.9)	11.2	**72.0±0.4**	**63.6±0.2**	+6.4%

(b) Pixel-Level Prediction

	Method (SegNet)	#P (M)	Segm. mIoU (↑)	Depth Abs. Err. (↓)	Δ_p (↑)
Cityscapes	Hard sharing	25.1	56.57±0.22	0.0170±0.0006	0.0%
	Attentive hard sharing [28]	28.2	55.45±1.03	0.0160±0.0006	+2.0%
	Task routing (γ =0.6) [37]	25.1	56.52±0.41	0.0155±0.0003	+4.4%
	Max roaming (γ =0.6) [30]	25.1	57.93±0.20	0.0143±0.0001	+9.1%
	ETR (γ =0.9)	25.1	**61.22±0.16**	**0.0141±0.0001**	+12.6%

Table 3 compares our explicit task routing with other parameter partitioning methods. All methods use regular convolution. We make the following observations, (i) All parameter partitioning methods improve performance over hard sharing. (ii) Our explicit task routing strategy is more effective for alleviating task interference and improving performance on both image-level classification and pixel-level dense prediction tasks. Additionally, to further understand the utility of explicit task routing, we visualize the features extracted by the shared branch and task-specific branches using t-SNE [39] in Figure 7. We observe that the shared branch extracts similar features across tasks, while the taskspecific stems extract individualized features for specific tasks. We also notice that the features extracted by taskspecific branches are dissimilar for different tasks, which shows that our explicit task routing can obtain task-specific features and mitigate mutual interference between tasks.

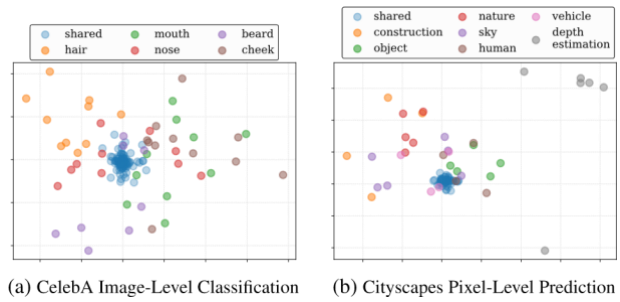


Figure 7. t-SNE visualization of feature activations from ETR's shared branch and task-specific branches of a single image on (a) CelebA and (b) Cityscapes. Note that features from the shared branch are clustered, while task-specific branches are spread-out.

4.2.3 ETR-NLP Networks

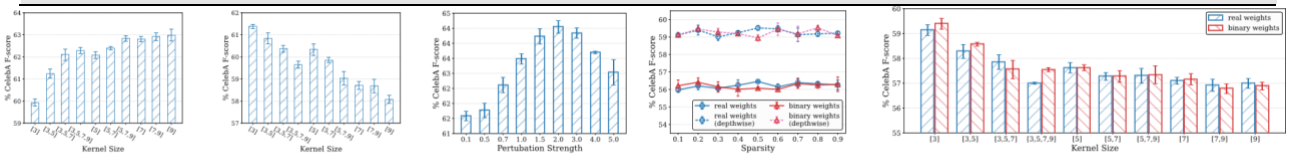
Table 4 presents the performance of various methods on the CelebA image-level classification problems. We consider two experimental settings, one with eight group-level tasks and another with 40 binary tasks. We make the following observations, (i) Our ETR-NLP is consistently better than the baselines methods while having fewer learnable parameters. (ii) We observe that the performance gains become more prominent as the number of tasks increases due to the inherent minimization of interference between tasks in ETR-NLP. (iii) We also observe that parameter partitioning methods (i.e., task routing, max roaming, and ETR-NLP) scale better than loss/gradient balancing methods (i.e., GradNorm and MGDA-UB) to a higher number of tasks. For instance, when the number of tasks increases from 8 to 40, the F-score of MGDA-UB decreases by 1.3%, while the F-score of our proposed ETR-NLP improves by 0.3% while having 28.5% fewer (11.2M to 8M) learnable parameters.

Table 4. Comparison of ETR-NLP with baselines on CelebA image-level classification problems. Our results are highlighted with shading.

Method (ResNet18)	#P (M)	8 grouped facial attributes (tasks) Precision (↑)	Recall (↑)	F-score (↑)	Δ_p (↑)	40 facial attributes (tasks) Precision (↑)	Recall (↑)	F-score (↑)	Δ_p (↑)
Hard sharing	11.2	67.7±0.8	59.8±0.3	63.0±0.2	0.0%	70.8±0.9	60.0±0.3	64.2±0.1	0.0%
GradNorm (α =0.5) [52]	11.2	70.4±0.1	59.5±0.6	63.6±0.5	+1.5%	70.7±0.8	60.0±0.3	64.1±0.3	-0.1%
MGDA-UB [34]	11.2	68.6±0.1	60.2±0.3	63.6±0.3	+1.0%	71.8±0.9	57.4±0.3	62.3±0.2	-2.0%
Atten. hard sharing [28]	12.9	71.1±0.3	62.6±0.5	65.9±0.2	+4.8%	73.2±0.1	63.6±0.2	67.5±0.1	+4.8%
Task routing [37]	11.2	71.7±0.1	61.7±0.5	65.5±0.3	+4.4%	72.1±0.8	63.4±0.3	66.8±0.2	+3.9%
Max roaming [30]	11.2	71.2±0.4	63.0±0.6	66.2±0.2	+5.2%	73.0±0.4	63.6±0.1	67.3±0.1	+4.6%
ETR-NLP	8.0	**72.7**±0.4	**64.8**±0.3	**67.8**±0.1	+7.8%	**73.2**±0.2	**64.8**±0.3	**68.1**±0.1	+5.8%

Table 5. Comparison of ETR-NLP with baselines on NYU-v2 pixel-level prediction problems. Our results are highlighted with shading.

Method (SegNet)	# P (M)	Segm. mIoU (↑)	Depth Estimation (Lower better ↓) Abs. Err.	Rel. Err.	Surface Normal Estimation Angle distance (↓) Mean Err.	Median Err.	Within t^o (↑) 11.25	22.5	30	Δ_p (↑)
Hard sharing	25.1	15.98±0.56	0.6095±0.0041	0.2554±0.0007	32.43±0.19	27.43±0.35	20.66±0.19	42.84±0.19	55.02±0.19	0.0%
GradNorm [52]	25.1	16.13±0.23	0.7626±0.0034	0.3208±0.0050	34.45±0.52	30.98±0.80	18.96±0.60	40.85±0.92	53.34±0.24	-10.6%
Cross-Stitch [29]	75.3	14.71±0.23	0.6481±0.0034	0.2871±0.0050	33.56±0.52	28.58±0.80	20.08±0.80	40.54±0.80	51.97±0.80	-6.0%
MTAN [22]	44.4	17.72±0.23	0.5960±0.0034	0.2577±0.0050	31.44±0.52	25.37±0.80	23.17±0.80	45.65±0.80	57.48±0.80	+5.7%
Atten. [28]	25.1	16.02±0.12	0.5988±0.0112	0.2630±0.0058	32.22±0.02	26.12±0.02	20.44±0.09	42.86±0.34	55.14±0.67	+0.5%
Task routing [37]	25.1	16.54±0.02	0.6354±0.0085	0.2786±0.0090	30.93±0.19	25.51±0.28	22.52±0.36	45.41±0.82	57.46±0.37	+2.7%
Max roaming [30]	25.1	17.40±0.31	0.6082±0.0023	0.2750±0.0015	30.58±0.04	24.67±0.08	23.74±0.61	46.75±0.41	58.84±0.28	+6.0%
ETR-NLP	25.1	**20.37**±0.32	**0.5790**±0.0067	**0.2510**±0.0090	**28.92**±0.05	**23.22**±0.16	**25.38**±0.11	**49.11**±0.27	**61.22**±0.23	+13.6%



(a) Avg. Pooling (b) Max Pooling (c) Perturbation (d) Convolution (sparsity) (e) Convolution (kernel size)

Figure 8. Effect of different hyperparameters of individual NLP.

Table 5 and Table 6 show the experimental results for pixel-level dense prediction problems on NYUv2 and Cityscapes datasets, respectively. Again, we observe that ETR-NLP significantly outperforms the baselines. Furthermore, it is worth mentioning that our ETR-NLP is the highest on all metrics. For instance, as shown in Table 6, ETRNLP obtains 61.49 mIoU for semantic segmentation, an improvement of +3.56 mIoU over the previous state-of-the-art results, while having 13.5% less learnable parameters. A similarly significant improvement is observed on the NYUv2 dataset across *all* tasks.

5. Ablation Analysis

NLP Hyperparameters: Figure 8 shows the effect of different settings for individual NLPs for image-level classification tasks. We observe that (i) A combination of different kernel sizes can improve performance even for the same type of NLP (e.g., avg pooling), as kernels of different sizes can extract diverse features. These results also indicate that multi-task learning benefits from operating on diverse features. (ii) For each type ofLP, the choice of parameters greatly impacts performance. For instance, for avg pooling, the F-score with a kernel size of 9 is 3% higher than that with a kernel size of 3.

The final design of our proposed NLP for image-level classification tasks was guided by the observations that we summarize as follows: (1) average pooling with larger kernels outperforms max pooling; (2) depth-wise convolutions outperform full convolutions while there is no appreciable difference between using real-valued or binary weights; (3) smaller convolution kernels outperform larger ones; (4) perturbation helps improve performance.

Training strategy for ETR-NLP: Table 7 shows the experimental results of “Steady-state” and “Synchronized” training strategies on the CelebA image-level and Cityscapes pixel-level prediction tasks. “Steady-state” refers to up-

Table 6. Comparison of ETR-NLP with baselines on Cityscapes pixel-level predictions. Our results are highlighted with shading.

Method (SegNet)	# P (M)	Segm. mIoU (↑)	Depth Estimation Abs. Err. (↓)	Rel. Err. (↓)	Δ_p (↑)

Hard sharing	25.1	56.57±0.22	0.0170±0.0006	43.99±5.53	0.0%
GradNorm [52]	25.1	56.77±0.08	0.0199±0.0004	68.13±4.48	- 23.9%
Cross-Stitch [29]	75.3	50.08±0.23	0.0154±0.0001	34.49±1.24	+6.5%
MTAN [22]	44.4	53.04±0.32	0.0144±0.0001	33.63±1.51	+10.9%
Atten. [28]	25.1	55.45±1.03	0.0160±0.0006	35.72±1.62	+7.6%
Task routing [37]	25.1	56.52±0.41	0.0155±0.0003	31.47±0.55	+12.4%
Max roaming [30]	25.1	57.93±0.20	0.0143±0.0001	29.38±1.66	+17.2%
		61.49	**0.0136**	**29.16**	**20.8%**
ETR-NLP	22.1	**±0.29**	**±0.0001**	**±1.30** +	

Table 7. Comparison of different training strategies for ETR on image-level classification and pixel-level prediction problems.

Method	CelebA		Cityscapes	
	Precision (↑)	Recall (↑)	mIoU (↑)	Abs. Err. (↓)
steady-state	72.0±0.4	63.6±0.2	59.70±0.38	0.0139±0.0002
synchronized	23.5±2.4	42.1±3.7	61.22±0.16	0.0141±0.0001

dating the parameters of the shared branch immediately after forwarding on one task, while "Synchronized" refers to waiting until all tasks are forwarded before updating the parameters of the shared branch. For image-level classification tasks, "steady-state" training is better, while for pixellevel prediction tasks, "synchronized" training is better.

6. Conclusion

In this paper, we present the Explicit Task Routing with Non-Learnable Primitives (ETR-NLP) module for multitask learning. The ETR-NLP module introduces nonlearnable primitives for extracting task-agnostic and diverse features and explicit task routing to extract task-specific features for each task. Both non-learnable primitives and explicit task routing can provide the flexibility needed to minimize task interference. Experiments on the CelebA dataset with multiple image-level classification tasks and on the NYU-v2 and Cityscapes datasets with multiple pixellevel prediction tasks show that our ETR-NLP method significantly outperforms state-of-the-art baselines with fewer learnable parameters and similar FLOPs across all datasets.

Acknowledgements

This work was supported by the National Natural Science Foundation of China (No. 62202039, 62106097, 62032003) and the National Key Research and Development Program of China (No. 2022ZD0118502). This appendix includes the following:

1. Extended description of Related Work in Section A.
2. Additional results of configurations on NLPs-based feature extraction in Section B.
3. Extended description of datasets and baseline methods in Section C.

A. An Extended Description of Related Work

Existing methods related to MTL architectures can be classified into encoder or decoder-focused ones. Encoderfocused approaches primarily lay emphasis on architectures that can encode multi-purpose feature representations through supervision from multiple tasks. Such encoding is typically achieved, for example, via feature fusion [11,26, 29], branching [18,24,25,40], self-supervision [7], shared and task-specific modules [22, 28], filter grouping [2], filter modulation [16, 53], task routing [26, 32, 37], or neural architecture search [10]. Decoder-focused approaches start from the feature representations learned at the encoding stage, and further refine them at the decoding stage by distilling information across tasks in a one-off [46], sequential [50], recursive [51], or even multi-scale [42] manner. Due to inherent layer sharing, the approaches above typically suffer from task interference and negative transfer [45].

In the context of MTL, our explicit task routing layer is conceptually related to [22, 28], but notably different in motivation and design. First, both of these two approaches operate on the features obtained from a shared backbone to extract task-specific features. In contrast, our shared and task-specific branches operate in parallel and extract features from the common features extracted by the nonlearnable layer. Second, these existing approaches utilize attention mechanisms to distill task-specific features from the shared features, while we use lightweight 1×1 convolution for the same purpose. Third, our explicit task routing layer is tailored to exploit the non-learnable layer for MTL optimally. Finally, unlike baselines, our multi-branch design affords simple and explicit control over the ratio of shared and task-specific parameters.

Additionally, our work is also closely related to reparameterized convolutions for multi-task learning (RCM) [16], which first introduced the concept of using non-learnable convolutional filters for MTL. However, there are three notable differences. First, the non-learnable layer of RCM only includes standard convolution, while we consider other non-learnable primitives such as pooling, identity, and additive noise [47] operations. Second, RCM uses pre-trained network weights to initialize non-learnable convolutional filters, while in our case they are sampled from a random distribution. Relying on pre-trained weights limits RCM's ability to reduce the model size and its generalizability to architectures without readily available pre-trained weights.

Table S1. Effect of different configurations of NLPs on CelebA multi-attribute classification.

Finally, there is no collaboration between tasks in RCM as it only comprises task-specific modulators, while we utilize a shared branch to help tasks use each other's training signals. Having both shared and task-specific branches

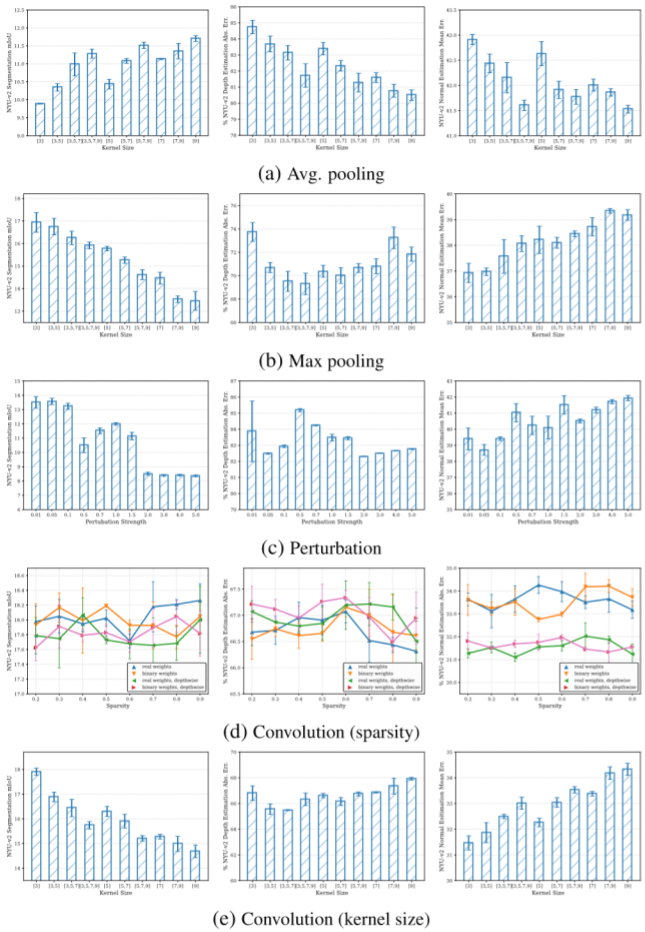


Figure S1. Effect of different hyperparameters of individual NLP on NYU-v2 dense prediction MTL problem. For each sub-figure (a) - (e), we show the semantic segmentation mIoU (↑), depth estimation absolute error (↓), and surface normal estimation mean error (↓).

allows tasks to amortize parameters that are commonly useful across multiple tasks, thereby minimizing redundancy in the task-specific branches, unlike RCM. Moreover, our method also offers fine-grained control over the ratio of parameters that are shared or task-specific.

B. Results of NLPs-based feature extraction

In this section, we first provide the full version of Table 1 from the main paper in Table S1, showing the effect of different configurations of NLPs on CelebA multi-attribute classification. Then, we present the effect of different hyperparameter settings of NLPs on NYU-v2 dense prediction MTL problem in Figure S1.

C. Description of Datasets and Baselines

In this section, we first provide the additional details of CelebA and Cityscapes datasets in Table 8 and 9, respectively. Then, we provide a brief overview of the baseline methods that we compared against in this work, as follows:

- STL: single task learning with one network for each task.
- Hard sharing: standard multi-task learning, i.e., a fully shared network with uniform task weighting.
- GradNorm [52]: a MTL method with a fully shared network and learnable tasks weighting.
- MGDA-UB [34]: a multi-objective alternative to MTL with a fully shared network.
- Task Routing [37]: a parameter partitioning method with randomly initialized binary masks.
- Max. Roaming [30]: another parameter partitioning method with dynamic masks.
- MTAN [22]: another soft-sharing method with attention.

Table 8. Details of the 40 attributes provided by the CelebA dataset [23]. For visualization purposes, we group them into eight categories.

Group	40-attribute
global	attractive, blurry, chubby, double chin, heavy makeup, male, oval face, pale skin, young
eyes	bags under eyes, eyeglasses, narrow eyes, arched eyebrows, bushy eyebrows
hair	bald, bangs, black hair, blond hair, brown hair, gray hair, receding hairline, straight hair, wavy hair
mouth	big lips, mouth slightly open, smiling, wearing lipstick
nose	big nose, pointy nose
beard	5 o'clock shadow, goatee, mustache, no beard, sideburns
cheek	high cheekbones, rosy cheeks
wearings	wearing earrings, wearing hat, wearing necklace, wearing necktie

Table 9. Three levels of semantic categories for the Cityscapes dataset [5, 22]. The experimental results in this work are based on the 7-class setting.

2-class	7-class	19-class
background	void	void
	flat	road, sidewalk
	construction	building, wall, fence
	object	pole, traffic light, traffic sign
	nature	vegetation, terrain
	sky	sky
foreground	human	person, rider
	vehicle	carm truck, bus, caravan, trailer, train, motorcycle

- Cross-stitch [29]: a soft-sharing method with feature fusion.

References

[1] V. Badrinarayanan, A. Kendall, and R. Cipolla. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(12):2481–2495, 2017. 3, 6

[2] Felix JS Bragman, Ryutaro Tanno, Sebastien Ourselin, Daniel C Alexander, and Jorge Cardoso. Stochastic filter groups for multi-task cnns: Learning specialist and generalist convolution kernels. In *International Conference on Computer Vision (ICCV)*, pages 1385–1394, 2019. 1, 3, 9

[3] Rich Caruana. Multitask learning. *Machine learning*, 28(1):41–75, 1997. 2, 6

[4] Weijie Chen, Di Xie, Yuan Zhang, and Shiliang Pu. All you need is a few shifts: Designing efficient convolutional neural networks for image classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7241–7250, 2019. 2, 3, 4, 6

[5] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3213–3223, 2016. 4, 5, 10

[6] Michael Crawshaw. Multi-task learning with deep neural networks: A survey, 2020. CoRR abs/2009.09796. 1

[7] Carl Doersch and Andrew Zisserman. Multi-task selfsupervised visual learning. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2070–2079, 2017. 9

[8] David Eigen and Rob Fergus. Predicting depth, surface normals and semantic labels with a common multi-scale convolutional architecture. In *International Conference on Computer Vision (ICCV)*, pages 2650–2658, 2015. 1, 3

[9] Adam Gaier and David Ha. Weight agnostic neural networks. In *Neural Information Processing Systems (NeurIPS)*, pages 5365–5379, 2019. 2

[10] Yuan Gao, Haoping Bai, Zequn Jie, Jiayi Ma, Kui Jia, and Wei Liu. Mtl-nas: Task-agnostic neural architecture search towards general-purpose multi-task learning. In *IEEE Conference on Computer Vision and pattern Recognition (CVPR)*, pages 11540–11549, 2020. 9

[11] Yuan Gao, Jiayi Ma, Mingbo Zhao, Wei Liu, and Alan L. Yuille. Nddr-cnn: Layerwise feature fusing in multi-task cnns by neural discriminative dimensionality reduction. In *IEEE Conference on Computer Vision and pattern Recognition (CVPR)*, pages 3205–3214, 2019. 9

[12] Kai Han, Yunhe Wang, Qi Tian, Jianyuan Guo, Chunjing Xu, and Chang Xu. Ghostnet: More features from cheap operations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1577–1586, 2020. 6

[13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 3, 6

[14] Caruana, Rich. "Multitask Learning." Machine Learning 28, no. 1 (1997): 41-75.

[15] Misra, Ishan, Abhinav Shrivastava, Abhinav Gupta, and Martial Hebert. "Cross-Stitch Networks for Multi-Task Learning." In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 3994-4003. 2016.

[16] Baxter, Jonathan. "A Bayesian/Informatical Model of Learning to Learn via Multiple Task Sampling." Machine Learning 28.1 (1997): 7-39.

[17] Ruder, Sebastian. "An Overview of Multi-Task Learning in Deep Neural Networks." arXiv preprint arXiv:1706.05098, 2017.

[18] Zhang, Yu, and Qiang Yang. "A Survey on Multi-task Learning." arXiv preprint arXiv:1707.08114, 2017.

[19] Bilen, Hakan, and Andrea Vedaldi. "Integrated Perceptual Object Detection." arXiv preprint arXiv:1606.04967, 2016.

[20] Perwein, Thomas, John A. Lee, et al. "Multi-task Learning for Attribute Prediction in the Wild." IEEE Pattern Analysis and Machine Intelligence, 2015.

[21] Collobert, Ronan, and Jason Weston. "A Unified Architecture for Natural Language Processing: Deep Neural Networks with Multitask Learning." In Proceedings of the 25th International Conference on Machine Learning, 160-167. ACM, 2008.

[22] Kokkinos, Iasonas. "UberNet: Training a Universal Convolutional Neural Network for Low-, Mid-, and High-Level Vision using Diverse Datasets and Limited Memory." In CVPR, 5454-5462. 2017.

[23] Darshan Bhavesh Mehta. (2024). NLP Enhanced IVR Routing for Electric Vehicle Fleet Operations Utilizing Solar Charging Infrastructure. International Journal of Communication Networks and Information Security (IJCNIS), 16(5), 978–991

[24] Alvarez, Josv© M, and Mathieu Salzmann. "Modeling and Selecting Multitask Learners." In ECCV, 2016.

[25] LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. "Deep Learning." Nature 521, no. 7553 (2015): 436-44.

[26] Simonyan, Karen, and Andrew Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition." In International Conference on Learning Representations, 2015.

[27] Szegedy, Christian, Wei Liu, Yangqing Jia, et al. "Going Deeper with Convolutions." In CVPR, 1-9. 2015.

[28] Long, Mingsheng, and Jianmin Wang. "Learning Multiple Tasks with Deep Relationship Networks." arXiv preprint arXiv: 1506.02117, 2015.

[29] Liu, Ziwei, Ping Luo, Xiaogang Wang, and Xiaoou Tang. "Deep Learning Face Attributes in the Wild." In ICCV, 3730-3738. 2015.

[30] SvТgaard, Anders, and Yoav Goldberg. "Deep Multi-Task Learning with Low Level Tasks Supervised at Lower Layers." In ACL, 231-235. 2016.

[31] Dosovitskiy, Alexey, Jost Tobias Springenberg, Martin A Riedmiller, and Thomas Brox. "Discriminative Unsupervised Feature Learning with Convolutional Neural Networks." In NeurIPS, 2014.

[32] Zhang, Yu, and Dit-Yan Yeung. "A Convex Formulation for Learning Task Relationships in Multi-Task Learning." In UAI, 733-742. 2010.

[33] Argyriou, Andreas, Theodoros Evgeniou, and Massimiliano Pontil. "Multi-task Feature Learning." In NeurIPS, 2006.

[34] Kumar, Abhishek, and Hal DaumV© III. "Learning Task Grouping and Overlap in Multi-Task Learning." In ICML, 1383-1390. 2012.

[35] Maurer, Andreas, Massimiliano Pontil, and Bernardino Romera-Paredes. "Sparse Coding for Multitask and Transfer Learning." In ICML, 343-351. 2013.

[36] Zhang, Shiliang, and Ming Li. "Multi-task Joint Learning Model for Chinese Word Usage Error Detection." In EMNLP, 694-703. 2015.

[37] Lu, Rongmei, Jiajun Bu, and Chun Chen. "Learning Task Dependencies for Robust Multi-Task Learning." In AAAI, 2015.

[38] Evgeniou, Theodoros, and Massimiliano Pontil. "Regularized Multi-task Learning." In KDD. 2004.

[39] Obozinski, Guillaume, Ben Taskar, and Michael Jordan. "Multi-task Feature Selection." arXiv preprint arXiv: 0811.0484. 2008.

[40] Ciliberto, Carlo, Lorenzo Rosasco, and Silvia Villa. "Symmetry and Invariance in Structured Prediction with Applications to Multi-Task Learning." NeurIPS, 2015.

[41] Ando, Rie K, and Tong Zhang. "A Framework for Learning Predictive Structures from Multiple Tasks and Unlabeled Data." Journal of Machine Learning Research 6 (2005): 1817-1853.

[42] Thrun, Sebastian, and Lorien Pratt. "Learning to Learn: Introduction and Overview." In Learning to Learn, 3-17. Springer, Boston, MA, 1998.

[43] Sayed, Ali H, and Dimitri Bertsekas. "Parameter and Hyperparameter Sharing in Neural Grid Learning." In Advances in Neural Information Processing Systems, 1135-1142. MIT Press, 1995.

[44] Xiangyun Zhao, Haoxiang Li, Xiaohui Shen, Xiaodan Liang, and Ying Wu. A modulation module for multi-task learning with applications in image retrieval. In *European Conference on Computer Vision (ECCV)*, pages 415–432, 2018. 1, 9